

RAPPORT DE PROJET — DEEP LEARNING

4^{ème} année Ingénierie AI /DATA

Apprentissage Profond appliqué à l'Agriculture Intelligente

Des réseaux denses à la prévision séquentielle : MLP, CNN et LSTM au service du monde agricole

Trois architectures, une mission commune

Recommandation de cultures | [Reconnaissance de chiffres](#) | [Prévision météorologique](#)

Réalisé par :

. AMYAR Tawhida

Encadré par :

. Mme. HIDILA Zineb

Table des matières

1	Introduction générale	1
1.1	Contexte du projet	1
1.2	Architectures étudiées	1
1.3	Synthèse des résultats	2
1.4	Organisation du rapport	2
2	Partie I — Perceptron Multicouche (MLP)	4
2.1	Présentation de la tâche et du jeu de données	4
2.2	Préparation des données	4
2.3	Architecture du modèle	5
2.3.1	Hyperparamètres d'entraînement	5
2.4	Étude de l'initialisation des poids	5
2.5	Résultats et courbes d'apprentissage	6
2.6	Matrice de confusion	7
2.7	Bilan de la Partie I	8
3	Partie II — Réseau de Neurones Convolutif (CNN)	9
3.1	Présentation de la tâche et du jeu de données	9
3.2	Architecture du modèle : LeNet modifié	9
3.2.1	Hyperparamètres d'entraînement	10
3.3	Courbes d'apprentissage	10
3.4	Visualisation des cartes de caractéristiques (<i>feature maps</i>)	11
3.5	Matrice de confusion	12
3.6	Comparaison MLP vs CNN	12
3.7	Bilan de la Partie II	13

4	Partie III — RNN, LSTM, GRU et Seq2Seq	14
4.1	Présentation de la tâche et du jeu de données	14
4.1.1	Construction des séquences	14
4.2	Comparaison des cellules récurrentes : RNN, LSTM, GRU	14
4.3	Architecture Seq2Seq (Encodeur-Décodeur)	15
4.4	Stratégies de décodage : Glouton vs Beam Search	16
4.5	Bilan de la Partie III	17
5	Discussion comparative	18
5.1	Synthèse transversale des trois architectures	18
5.2	Régularisation : un fil conducteur commun	19
5.3	De la donnée statique à la donnée séquentielle	19
6	Conclusion générale	20
6.1	Bilan du projet	20
	Annexes	24

Liste des figures

2.1	Comparaison de la perte de validation selon la stratégie d'initialisation (Xavier, Gaussienne, Constante).....	6
2.2	Courbes d'apprentissage du MLP (perte et précision) sur la détection de cultures agricoles	6
2.3	Matrice de confusion du MLP sur les 22 classes de cultures (ensemble de test).....	7
3.1	Exemples d'images du jeu de données MNIST avec leur classe associée	9
3.2	Courbes d'apprentissage du CNN LeNet sur MNIST (perte et précision).....	10
3.3	Cartes de caractéristiques après la première couche convolutive (Conv1) pour un chiffre "7"	11
3.4	Cartes de caractéristiques après la deuxième couche convolutive (Conv2) pour le même chiffre	11
3.5	Matrice de confusion du CNN sur les 10 classes de chiffres MNIST (ensemble de test)	12
3.6	Comparaison de la précision de validation entre un MLP simple et le CNN LeNet sur MNIST.....	13
4.1	Comparaison de la perte de validation (MSE) et de la perplexité entre RNN, LSTM et GRU	15
4.2	Comparaison des prévisions à 24h obtenues par décodage glouton et Beam Search ($k = 3$), sur deux exemples du jeu de test.....	16

Liste des tableaux

1.1	Vue d'ensemble des architectures et tâches du projet.....	2
1.2	Synthèse des performances finales	2
2.1	Hyperparamètres d'entraînement du MLP.....	5
3.1	Hyperparamètres d'entraînement du CNN.....	10
5.1	Comparaison transversale des trois architectures	18
6.1	Récapitulatif final des résultats et enseignements	20

Chapitre 1

Introduction générale

1.1 Contexte du projet

L'agriculture moderne fait face à des défis croissants : optimisation des rendements, anticipation des conditions climatiques et diagnostic rapide des cultures. Ce projet explore comment trois grandes familles de réseaux de neurones profonds peuvent répondre à ces besoins, en couvrant un spectre complet de l'apprentissage profond : des données tabulaires structurées à la prévision de séries temporelles, en passant par la vision par ordinateur.

Le fil conducteur retenu est résolument agricole : aider un agriculteur à choisir la culture la plus adaptée à son sol, démontrer la puissance de la vision par ordinateur sur un cas d'école reconnu (MNIST), puis anticiper les conditions météorologiques pour mieux préparer les prochaines récoltes.

Objectif pédagogique

Comparer trois paradigmes d'architectures profondes (perceptron multicouche, réseau convolutif, réseau récurrent) sur trois tâches distinctes, en mettant l'accent sur la rigueur expérimentale : initialisation des poids, régularisation, choix d'optimiseur et stratégies de décodage.

1.2 Architectures étudiées

Le tableau 1.1 résume les trois architectures développées, leur jeu de données respectif et la tâche associée.

Tableau 1.1: Vue d'ensemble des architectures et tâches du projet

Partie	Architecture	Jeu de données	Tâche
Partie I	MLP (Perceptron multicouche)	Crop Recommendation (2 200 ex., 22 classes)	Recommandation de culture
Partie II	CNN (LeNet modifié)	MNIST (70 000 images 28×28)	Classification de chiffres manuscrits
Partie III	RNN / LSTM / GRU / Seq2Seq	Jena Climate (420 551 obs. horaires)	Prévision météorologique à 24h

1.3 Synthèse des résultats

Le tableau 1.2 présente les performances finales obtenues pour chaque architecture sur son jeu de test respectif.

Tableau 1.2: Synthèse des performances finales

Modèle	Métrique principale	Validation	Test
MLP	Accuracy (22 classes)	$\approx 98,5\%$	$\approx 98\%$
CNN LeNet	Accuracy (MNIST)	$\approx 98,2\%$	$\approx 98,1\%$
Seq2Seq (GRU)	MSE température normalisée	$\approx 9,2 \times 10^{-4}$	—

1.4 Organisation du rapport

Le rapport est structuré en trois chapitres techniques correspondant aux trois parties du projet, suivis d'une discussion comparative transversale et d'une conclusion générale.

- Le **Chapitre 2** présente le perceptron multicouche appliqué à la recommandation de culture, avec une étude approfondie de l'effet de l'initialisation des poids.
- Le **Chapitre 3** détaille le réseau convolutif de type LeNet sur MNIST, avec visualisation des cartes de caractéristiques (*feature maps*) et comparaison directe avec un MLP.
- Le **Chapitre 4** explore les architectures récurrentes (RNN, LSTM, GRU) et un modèle Seq2Seq pour la prévision météorologique, avec une comparaison des stratégies de décodage glouton et *beam search*.
- Le **Chapitre 5** propose une synthèse comparative transversale des trois approches.

- Le rapport se conclut par un bilan général et des pistes d'amélioration (Chapitre 6).

Partie I — Perceptron Multicouche (MLP)

2.1 Présentation de la tâche et du jeu de données

La première partie du projet aborde une tâche de classification sur données tabulaires : recommander la culture la plus adaptée à partir de caractéristiques agronomiques du sol. Le jeu de données *Crop Recommendation Dataset* contient 2 200 exemples répartis en 22 classes de cultures (riz, maïs, coton, banane, café, etc.), chacun décrit par 7 variables numériques : teneur en azote (N), phosphore (P) et potassium (K), température, humidité, pH du sol et précipitations.

Pourquoi un MLP ici ?

Les données sont purement tabulaires, sans structure spatiale ni séquentielle : un perceptron multicouche, qui traite chaque caractéristique indépendamment via des combinaisons linéaires successives, est l'architecture la plus naturelle pour ce type de problème.

2.2 Préparation des données

Les données suivent un protocole standard de préparation avant entraînement :

1. **Encodage de la cible** : les 22 labels textuels sont convertis en entiers via un `LabelEncoder`.
2. **Division train/validation/test** : split stratifié 70 % / 15 % / 15 %, garantissant une représentation équilibrée des 22 classes dans chaque sous-ensemble.
3. **Normalisation** : standardisation (`StandardScaler`) ajustée uniquement sur l'ensemble d'entraînement puis appliquée à la validation et au test, afin d'éviter toute fuite d'information (*data leakage*).

2.3 Architecture du modèle

Architecture MLP retenue

- **Couche d'entrée** : 7 neurones (une par variable agronomique)
- **Couche cachée 1** : 256 neurones → BatchNorm → ReLU → Dropout (0,3)
- **Couche cachée 2** : 128 neurones → BatchNorm → ReLU → Dropout (0,3)
- **Couche de sortie** : 22 neurones (un par classe de culture)
- **Paramètres entraînaibles** : 38 550

La *Batch Normalization* après chaque couche linéaire stabilise les statistiques internes du réseau et accélère la convergence, tandis que le *Dropout* à 0,3 limite le surapprentissage en désactivant aléatoirement 30 % des neurones à chaque passage d'entraînement.

2.3.1 Hyperparamètres d'entraînement

Tableau 2.1: Hyperparamètres d'entraînement du MLP

Paramètre	Valeur
Optimiseur	Adam (weight decay 10^{-4})
Taux d'apprentissage initial	10^{-3}
Scheduler	ReduceLROnPlateau (patience 5, facteur 0,5)
Fonction de perte	CrossEntropyLoss
Taille de batch	64
Nombre d'époques	60 (avec early stopping)

2.4 Étude de l'initialisation des poids

Un volet clé de cette partie est la comparaison de trois stratégies d'initialisation des poids, toutes appliquées au même modèle et entraînées dans les mêmes conditions :

- **Xavier (Glorot)** : tirage uniforme dont la variance est calibrée selon la taille des couches d'entrée/sortie, conçue pour préserver la variance des activations à travers les couches profondes.
- **Gaussienne** : poids tirés d'une loi normale $N(0, 0,01)$, sans adaptation à la taille des couches.
- **Constante** : tous les poids initialisés à 1 — une pratique volontairement défaillante, incluse à titre pédagogique pour illustrer ses effets négatifs.

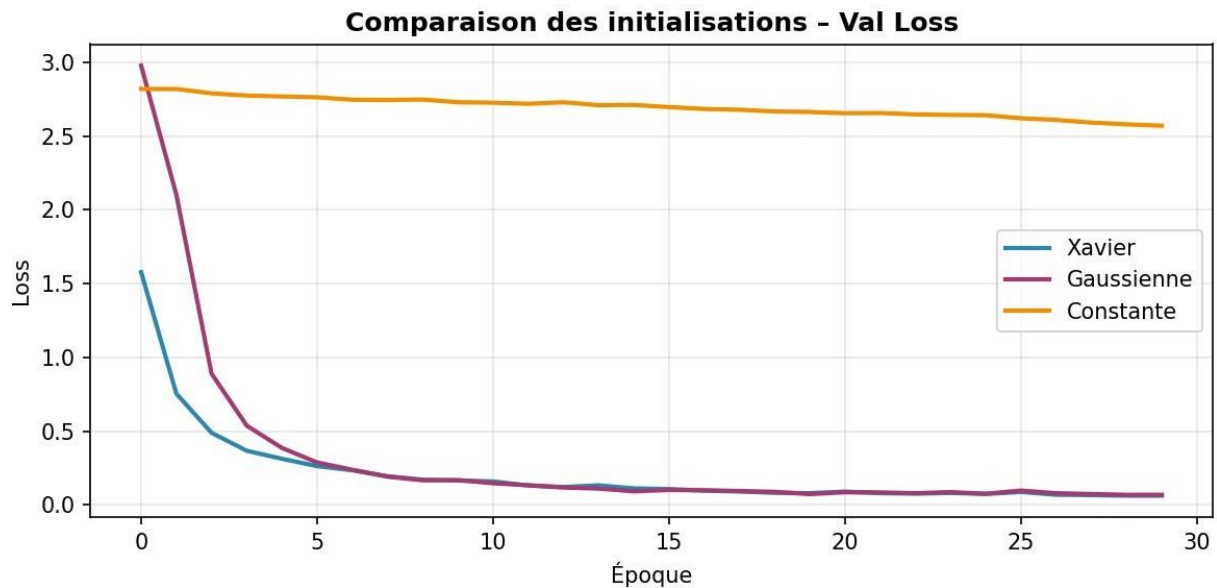


Figure 2.1: Comparaison de la perte de validation selon la stratégie d'initialisation (Xavier, Gaussienne, Constante)

Analyse de l'initialisation

La figure ci-dessus montre que l'initialisation **Xavier** converge le plus rapidement dès les premières époques, suivie de près par l'initialisation **Gaussienne** qui rattrape progressivement Xavier après une dizaine d'époques. L'initialisation **Constante**, en revanche, reste bloquée à une perte élevée ($\approx 2,6$) même après 30 époques : tous les neurones d'une même couche reçoivent un gradient identique (symétrie des poids), ce qui empêche le réseau d'apprendre des représentations différenciées — un cas d'école illustrant l'importance d'une initialisation brisant la symétrie.

2.5 Résultats et courbes d'apprentissage

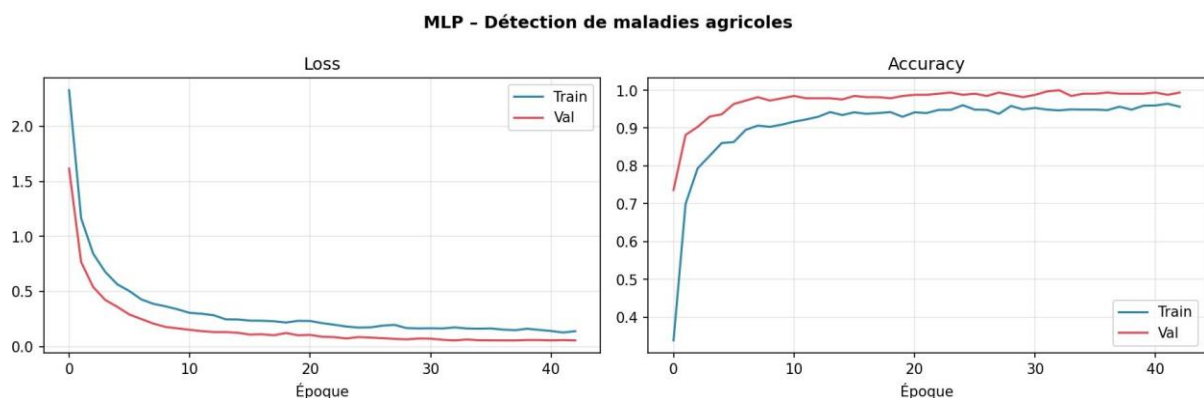


Figure 2.2: Courbes d'apprentissage du MLP (perte et précision) sur la détection de cultures agricoles

Les courbes d'apprentissage confirment une convergence saine : la perte de validation diminue de concert avec celle d'entraînement sans divergence notable, et la précision de validation dépasse 98 % dès la dixième époque, se stabilisant ensuite autour de 98,5 %. L'écart modéré

entre les courbes d’entraînement et de validation suggère une régularisation efficace, sans surapprentissage marqué.

2.6 Matrice de confusion

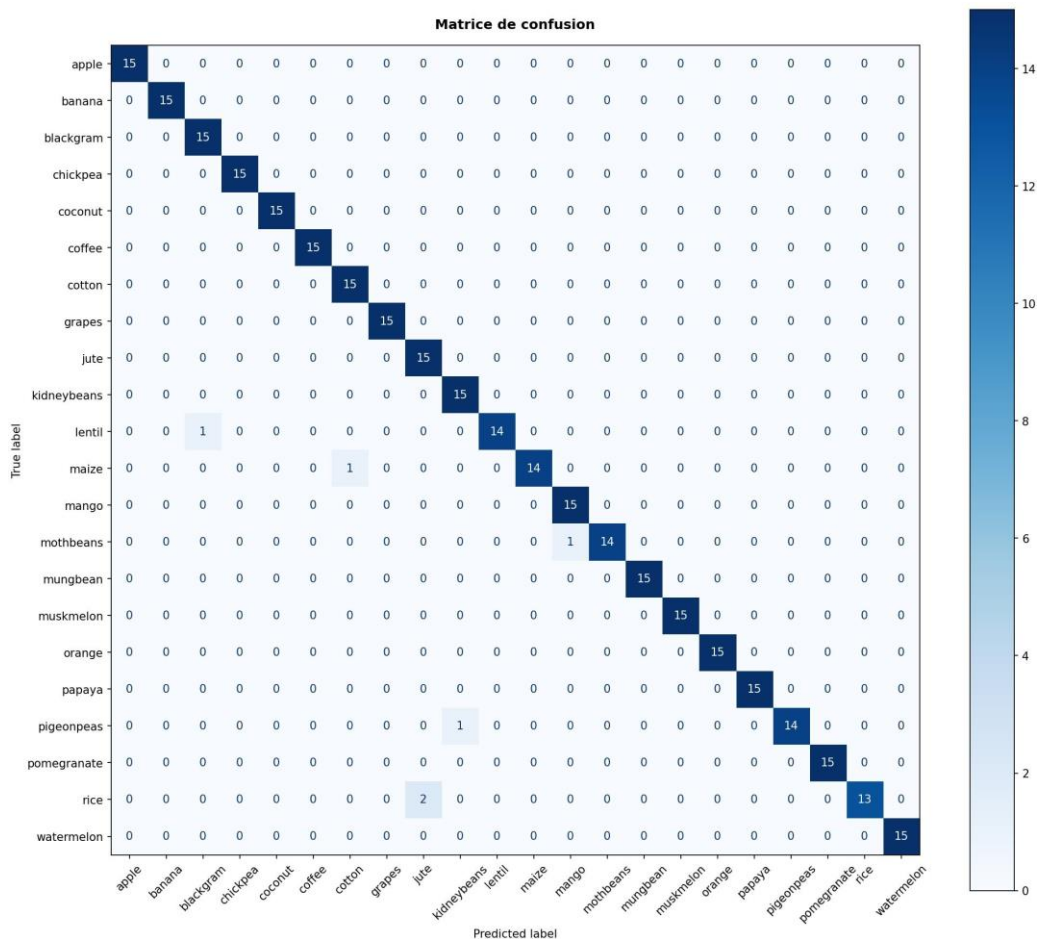


Figure 2.3: Matrice de confusion du MLP sur les 22 classes de cultures (ensemble de test)

La matrice de confusion révèle une classification quasi parfaite : la quasi-totalité des classes affichent une diagonale pure (aucune erreur). Les rares confusions concernent des classes agronomiquement proches, comme *rice* confondu deux fois avec *jute* (cultures partageant des besoins similaires en humidité et précipitations), ou *maize* confondu une fois avec *cotton*. Ces erreurs résiduelles, peu nombreuses, s’expliquent par le chevauchement naturel des profils agronomiques de certaines cultures plutôt que par une faiblesse du modèle.

2.7 Bilan de la Partie I

Synthèse

Le MLP, malgré sa simplicité architecturale, atteint une précision proche de 98,5 % sur un problème à 22 classes grâce à une combinaison efficace de BatchNorm, Dropout et d'une initialisation Xavier. L'étude comparative des initialisations met en évidence un principe fondamental du deep learning : une architecture performante peut être totalement neutralisée par un choix d'initialisation inadapté.

Partie II — Réseau de Neurones Convolutif (CNN)

3.1 Présentation de la tâche et du jeu de données

La deuxième partie aborde la vision par ordinateur à travers le jeu de données **MNIST**, référence historique du domaine, composé de 70 000 images en niveaux de gris de 28×28 pixels représentant des chiffres manuscrits de 0 à 9.

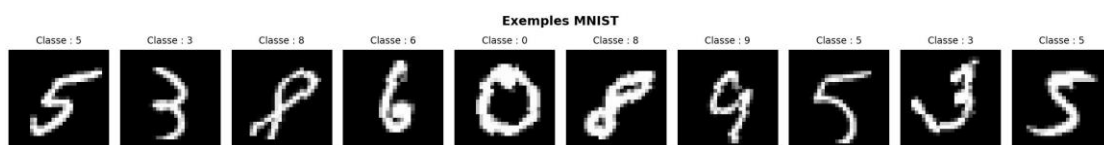


Figure 3.1: Exemples d'images du jeu de données MNIST avec leur classe associée

Pourquoi un CNN ici ?

Contrairement aux données tabulaires de la Partie I, une image possède une structure spatiale locale : les pixels voisins sont corrélés. Les couches convolutives exploitent cette structure en apprenant des filtres locaux réutilisables sur toute l'image, ce qu'un MLP ne peut faire qu'au prix d'un nombre de paramètres bien plus important.

3.2 Architecture du modèle : LeNet modifié

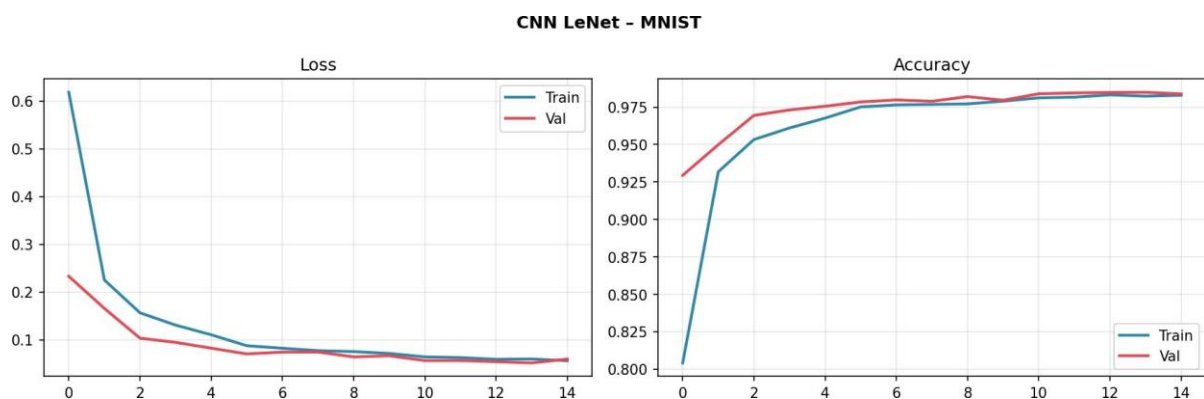
L'architecture retenue s'inspire du historique **LeNet-5** (LeCun et al., 1998), avec quelques adaptations modernes (Batch Normalization, Dropout, couche 1×1).

Architecture LeNet modifiée

- **Conv1** : Conv2D($1 \rightarrow 6$, noyau 5×5 , padding 2) $\rightarrow$ BatchNorm $\rightarrow$ Sigmoid $\rightarrow$ AvgPool (2×2) [$28 \times 28 \rightarrow 14 \times 14$]
- **Conv2** : Conv2D($6 \rightarrow 16$, noyau 5×5) $\rightarrow$ BatchNorm $\rightarrow$ Sigmoid $\rightarrow$ AvgPool (2×2) [$14 \times 14 \rightarrow 5 \times 5$]
- **Conv 1×1** : Conv2D($16 \rightarrow 16$, noyau 1×1) $\rightarrow$ ReLU — réduction/recombinaison des canaux sans coût spatial
- **Couches denses** : Flatten ($16 \times 5 \times 5 = 400$) $\rightarrow$ FC(120) $\rightarrow$ ReLU $\rightarrow$ Dropout (0,3) $\rightarrow$ FC(84) $\rightarrow$ ReLU $\rightarrow$ Dropout (0,3) $\rightarrow$ FC(10)
- **Paramètres entraînaibles** : 62 054

3.2.1 Hyperparamètres d'entraînement*Tableau 3.1: Hyperparamètres d'entraînement du CNN*

Paramètre	Valeur
Optimiseur	Adam (weight decay 10^{-4})
Taux d'apprentissage initial	10^{-3}
Scheduler	StepLR (pas de 5, facteur 0,5)
Fonction de perte	CrossEntropyLoss
Nombre d'époques	15

3.3 Courbes d'apprentissage*Figure 3.2: Courbes d'apprentissage du CNN LeNet sur MNIST (perte et précision)*

La convergence est rapide et stable : la perte de validation chute fortement dès les deux premières époques puis se stabilise sous 0,05, tandis que la précision de validation franchit 97 % dès la troisième époque pour finir au-delà de 98 %. L'absence d'écart significatif entre les courbes d'entraînement et de validation confirme une bonne généralisation, sans surapprentissage.

3.4 Visualisation des cartes de caractéristiques (*feature maps*)

L'un des grands avantages des CNN est leur interprétabilité partielle : il est possible de visualiser ce que chaque filtre convolutif "voit" dans l'image d'entrée.

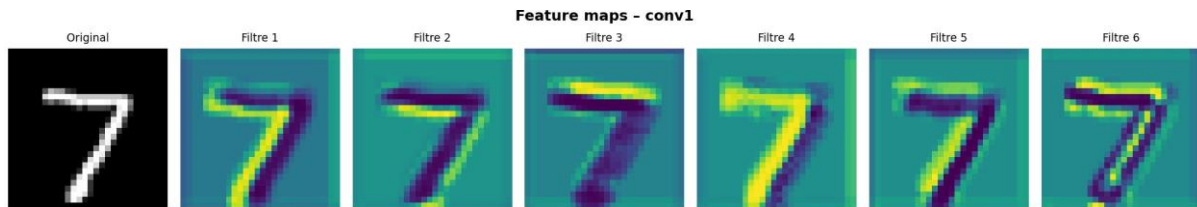


Figure 3.3: Cartes de caractéristiques après la première couche convolutive (Conv1) pour un chiffre "7"

Au niveau de **Conv1**, les filtres détectent des motifs visuels de bas niveau : contours, bords et orientations des traits du chiffre. On distingue nettement, selon le filtre, une réponse forte sur la barre horizontale supérieure du "7" (Filtres 1, 2, 5) ou sur sa diagonale descendante (Filtres 4, 6), chaque filtre se spécialisant dans une orientation différente.

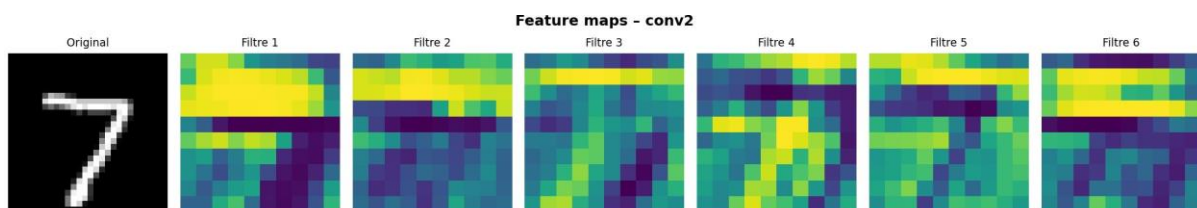


Figure 3.4: Cartes de caractéristiques après la deuxième couche convolutive (Conv2) pour le même chiffre

Après **Conv2**, la résolution spatiale est réduite (5×5) mais les représentations deviennent plus abstraites : les filtres combinent les motifs locaux détectés par Conv1 en caractéristiques de plus haut niveau, moins interprétables visuellement mais plus discriminantes pour la classification finale.

Hiérarchie des représentations

Cette progression illustre le principe fondamental des CNN : les couches profondes construisent une hiérarchie de représentations, des contours simples (Conv1) vers des motifs abstraits combinant plusieurs indices visuels (Conv2), jusqu'à la décision finale de classification.

3.5 Matrice de confusion

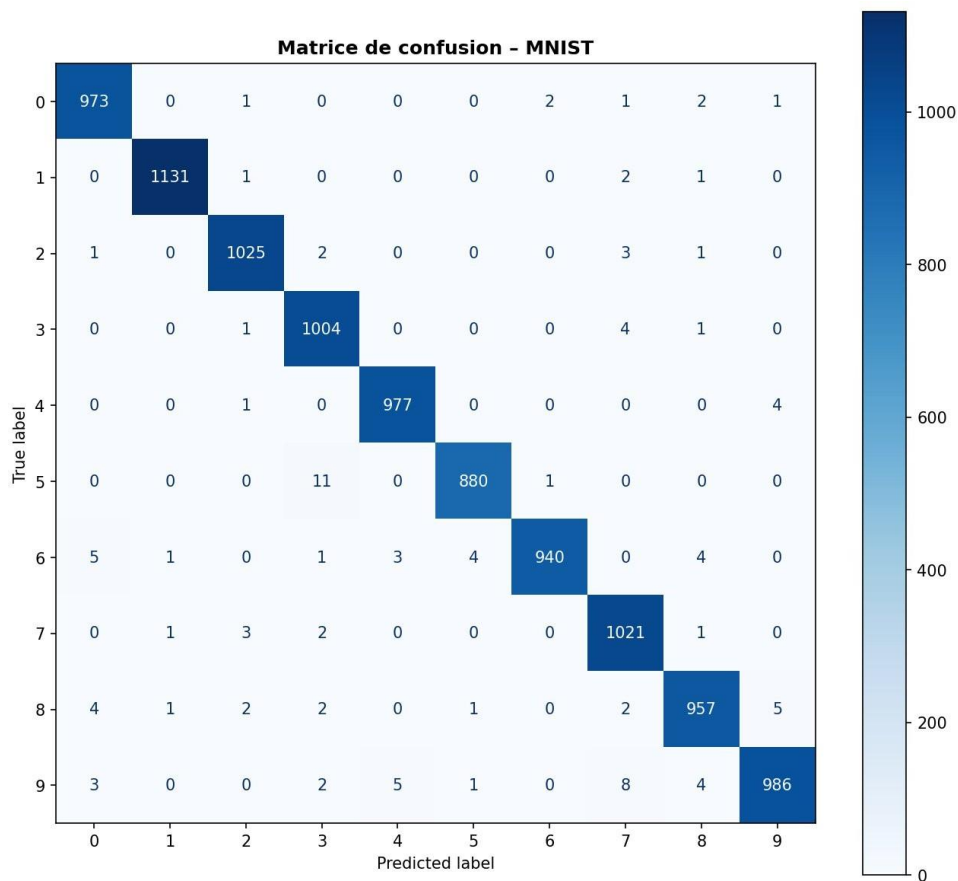


Figure 3.5: Matrice de confusion du CNN sur les 10 classes de chiffres MNIST (ensemble de test)

La matrice de confusion affiche une diagonale extrêmement dominante avec plus de 97 % de bonnes classifications par classe. Les confusions résiduelles les plus notables concernent la paire (5, 3) avec 11 erreurs, ainsi que des confusions isolées entre 9 et 4/7/8 — des paires de chiffres dont les tracés manuscrits peuvent effectivement se ressembler visuellement (boucles ouvertes, traits obliques similaires).

3.6 Comparaison MLP vs CNN

Pour quantifier l'apport de la convolution par rapport à un simple réseau dense, un MLP de référence (Flatten → FC(256) → FC(128) → FC(10)) a été entraîné sur les mêmes images MNIST, dans les mêmes conditions.

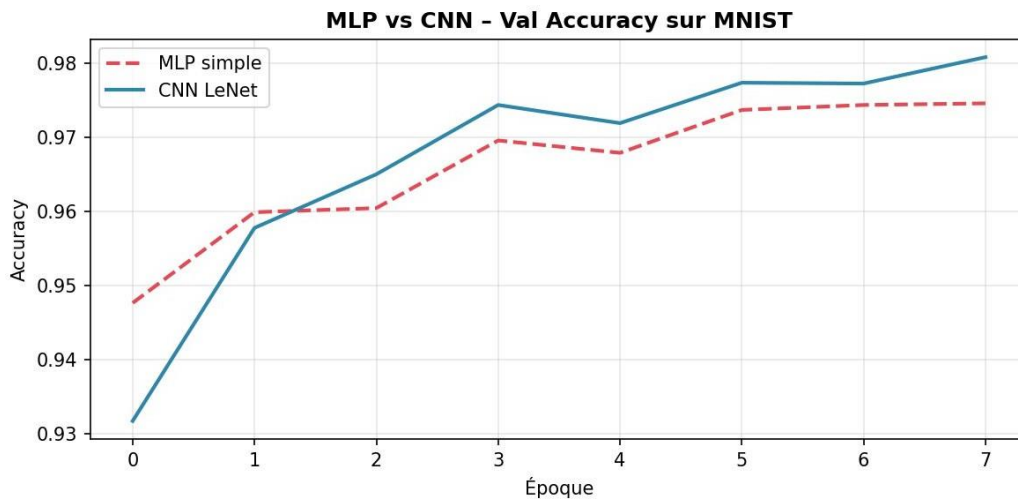


Figure 3.6: Comparaison de la précision de validation entre un MLP simple et le CNN LeNet sur MNIST

Analyse comparative

Sur les premières époques, le MLP converge initialement plus vite (il atteint 96 % de précision dès l'époque 1, contre 95,8 % pour le CNN). Cependant, à partir de l'époque 3, le CNN prend l'avantage et le conserve jusqu'à la fin de l'entraînement, terminant à 98,1 % contre 97,5 % pour le MLP. Ce résultat illustre que l'exploitation explicite de la structure spatiale de l'image, via les couches convolutives, permet au CNN d'extraire des caractéristiques plus pertinentes et donc d'atteindre une meilleure précision finale, malgré un nombre comparable de paramètres entraînaables.

3.7 Bilan de la Partie II

Le CNN LeNet modifié atteint une précision de test d'environ 98,1 % sur MNIST, surpassant le MLP de référence. La visualisation des cartes de caractéristiques confirme que le réseau apprend une hiérarchie cohérente de représentations, des contours simples vers des motifs discriminants de haut niveau, justifiant le choix architectural des couches convolutives pour les données visuelles.

Chapitre 4

Partie III — RNN, LSTM, GRU et Seq2Seq

4.1 Présentation de la tâche et du jeu de données

La troisième partie s'attaque à la prévision de séries temporelles, avec une application directe à l'agriculture : anticiper l'évolution météorologique pour alerter l'agriculteur. Le jeu de données **Jena Climate** regroupe 420 551 observations horaires (température, humidité relative, pression atmosphérique, vitesse du vent, précipitations) collectées entre 2009 et 2016 en Allemagne.

Pourquoi un réseau récurrent ici ?

Une série temporelle météorologique possède une dépendance séquentielle : la température à l'instant t dépend fortement des valeurs aux instants précédents. Les architectures récurrentes (RNN, LSTM, GRU) maintiennent un état caché qui résume l'historique de la séquence, les rendant naturellement adaptées à ce type de dépendance temporelle, contrairement aux MLP et CNN qui traitent les entrées de façon statique.

4.1.1 Construction des séquences

Les données brutes sont normalisées (min-max) puis découpées en fenêtres glissantes : chaque exemple est constitué d'une fenêtre de **72 heures passées** (variables d'entrée) utilisée pour prédire la **température des 24 heures suivantes**. Le jeu obtenu est divisé selon un ratio 70 / 15 / 15 % en respectant l'ordre chronologique (train, validation, test).

4.2 Comparaison des cellules récurrentes : RNN, LSTM, GRU

Trois variantes de cellules récurrentes ont été entraînées dans des conditions strictement identiques (2 couches, 128 unités cachées, dropout 0,2, *gradient clipping* à 1,0) pour comparer leur capacité à modéliser la dynamique météorologique.

Architecture WeatherRNN

- **Cellule récurrente** : RNN simple, LSTM ou GRU — 2 couches empilées, 128 unités cachées, dropout inter-couches 0,2
- **Couche de sortie** : Linear(128 → 24) appliquée au dernier état caché de la séquence
- **Entraînement** : Adam ($\text{lr}=10^{-3}$, weight decay 10^{-5}), *gradient clipping* (norme max 1,0), scheduler ReduceLROnPlateau, perte MSE

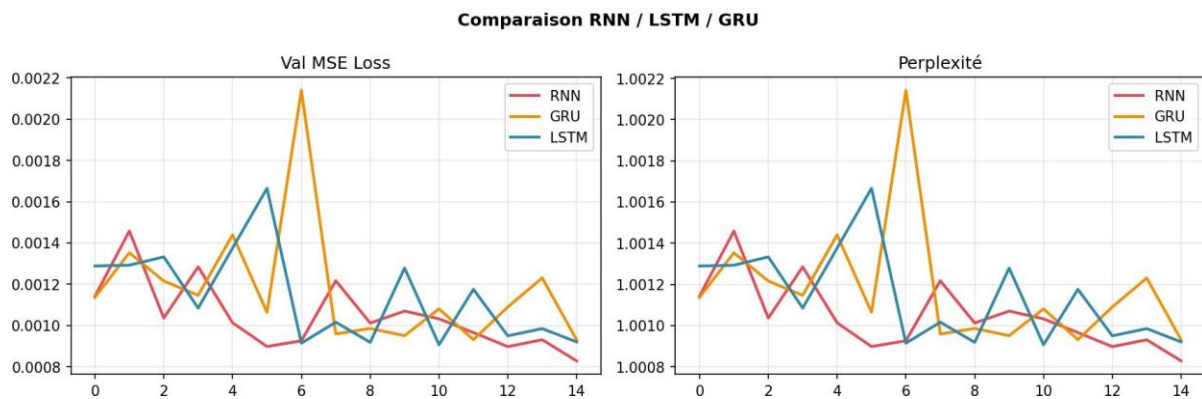


Figure 4.1: Comparaison de la perte de validation (MSE) et de la perplexité entre RNN, LSTM et GRU

Analyse comparative des cellules

Le **RNN simple** affiche la convergence la plus stable et atteint la perte de validation la plus basse en fin d'entraînement, suivi par le **LSTM**. Le **GRU** présente une instabilité notable autour de l'époque 6 (pic de perte), probablement liée à un pas d'apprentissage temporairement trop agressif pour cette architecture sur cette tâche, avant de converger à nouveau. Sur cet horizon de prévision relativement court (72h → 24h) et ce volume de données, le RNN simple — malgré sa réputation de sensibilité au problème de gradient évanescant sur de longues séquences — reste compétitif, les bénéfices du LSTM et du GRU se manifestant généralement davantage sur des séquences nettement plus longues.

4.3 Architecture Seq2Seq (Encodeur-Décodeur)

Pour générer une séquence de 24 prédictions de façon plus structurée qu'une simple couche dense finale, un modèle **Seq2Seq** de type encodeur-décodeur a été implémenté, suivant le paradigme introduit par Sutskever et al. (2014) et Cho et al. (2014).

Architecture Seq2Seq

- **Encodeur** : GRU (2 couches, 128 unités) qui compresse la fenêtre de 72h en un état caché final
- **Décodeur** : GRU à 128 unités, génère les 24 prédictions **pas à pas** ; chaque sortie devient l'entrée du pas suivant (génération auto-régressive)
- **Couche de projection** : Linear(128 → 1) appliquée à chaque pas du décodeur

Cette architecture permet, contrairement à l'approche directe précédente, de modéliser explicitement les dépendances entre les pas de temps prédits successifs.

4.4 Stratégies de décodage : Glouton vs Beam Search

Une fois le modèle Seq2Seq entraîné, deux stratégies de génération de séquence ont été comparées :

- **Décodage glouton (*greedy*)** : à chaque pas, le décodeur choisit directement sa propre prédiction comme entrée du pas suivant, sans explorer d'alternatives.
- **Beam Search ($k = 3$)** : à chaque pas, les 3 meilleures hypothèses partielles sont conservées et étendues en parallèle, la séquence finale retenue étant celle de meilleur score cumulé.

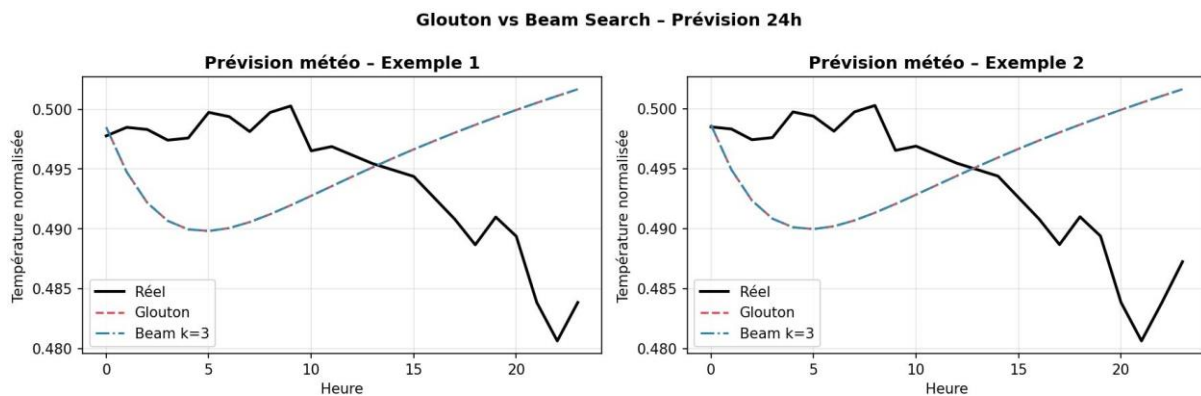


Figure 4.2: Comparaison des prévisions à 24h obtenues par décodage glouton et Beam Search ($k = 3$), sur deux exemples du jeu de test

Analyse du décodage

Sur les deux exemples présentés, les courbes de décodage glouton et de Beam Search sont quasiment superposées : pour une tâche de régression continue à sortie unique par pas de temps (contrairement à la génération de texte où plusieurs tokens sont possibles à chaque position), l'espace des choix à chaque étape est réduit à une seule valeur prédite déterministe, ce qui limite mécaniquement l'intérêt du Beam Search par rapport à son usage classique en traitement du langage naturel (où il explore plusieurs candidats discrets). Les deux stratégies suivent une tendance lissée qui s'écarte progressivement de la série réelle (courbe noire) sur l'horizon de 24h, illustrant la difficulté croissante de la prévision à mesure que l'horizon temporel s'éloigne du dernier point connu.

4.5 Bilan de la Partie III

Synthèse

Cette partie illustre la richesse des architectures récurrentes pour la modélisation séquentielle : la comparaison RNN/LSTM/GRU souligne que le choix de la cellule récurrente optimale dépend fortement de la tâche et de l'horizon temporel considéré, tandis que l'architecture Seq2Seq démontre l'intérêt d'un décodage auto-régressif structuré pour la génération de séquences multi-pas. La comparaison glouton/Beam Search met enfin en lumière une limite importante du Beam Search sur les tâches de régression continue, où son avantage habituel en NLP ne se manifeste pas de la même manière.

Chapitre 5

Discussion comparative

5.1 Synthèse transversale des trois architectures

Chacune des trois architectures étudiées a démontré son adéquation à la structure spécifique de son jeu de données : le MLP pour des vecteurs de caractéristiques tabulaires indépendantes, le CNN pour des grilles de pixels avec corrélations spatiales locales, et les réseaux récurrents pour des séquences temporelles ordonnées.

Tableau 5.1: Comparaison transversale des trois architectures

Critère	MLP	CNN	RNN/LSTM/GRU
Type de données	Tabulaire	Image (grille 2D)	Séquence temporelle
Hypothèse structurelle	Aucune	Localité spatiale	Ordre temporel
Partage de paramètres	Non	Oui (filtres convolutifs)	Oui (poids récurrents)
Paramètres entraîables	38 550	62 054	~260 000 (LSTM, 2 couches, 128 unités)
Principal risque	Surapprentissage si peu de régularisation	Coût calcul croissant avec la profondeur	Gradient évanescent / explosif sur longues séquences
Mécanisme clé observé	Sensibilité critique à l'initialisation des poids	Hiérarchie de représentations (Conv1 → Conv2)	Stabilité du RNN simple sur séquences courtes

5.2 Régularisation : un fil conducteur commun

Les trois parties du projet partagent un souci constant de régularisation, bien que les mécanismes diffèrent :

- Le **MLP** et le **CNN** combinent *Batch Normalization* et *Dropout* pour stabiliser l'entraînement et limiter le surapprentissage.
- Les réseaux **récurrents** utilisent le *gradient clipping*, indispensable pour éviter l'explosion des gradients caractéristique de la rétropropagation à travers le temps (*BPTT*).
- Dans les trois cas, un *scheduler* de taux d'apprentissage (*ReduceLROnPlateau* ou *StepLR*) ajuste dynamiquement la vitesse de convergence.

5.3 De la donnée statique à la donnée séquentielle

Un parallèle intéressant peut être tracé entre les trois parties : le MLP traite l'information de façon totalement statique (aucune notion d'ordre entre les 7 variables d'entrée), le CNN introduit une notion d'ordre *local* à travers la structure spatiale 2D de l'image, et les réseaux récurrents généralisent cette notion d'ordre à une dimension *globale et temporelle*. Cette progression illustre comment le choix architectural doit avant tout refléter la structure intrinsèque des données, plutôt que de privilégier systématiquement l'architecture la plus complexe.

Enseignement central du projet

Il n'existe pas d'architecture universellement supérieure : la performance d'un modèle de deep learning dépend avant tout de l'adéquation entre la structure de son architecture et la structure intrinsèque des données qu'il traite.

Conclusion générale

6.1 Bilan du projet

Ce projet a permis d'explorer trois grandes familles d'architectures de deep learning à travers un fil conducteur agricole cohérent : la recommandation de culture (MLP), la reconnaissance visuelle (CNN), et la prévision météorologique (LSTM/Seq2Seq). Chaque partie a été l'occasion d'appliquer et de valider des bonnes pratiques fondamentales du domaine : normalisation rigoureuse des données, choix réfléchi de l'initialisation des poids, régularisation par Dropout et Batch Normalization, et suivi attentif des courbes d'apprentissage pour détecter tout signe de surapprentissage.

Tableau 6.1: Récapitulatif final des résultats et enseignements

Partie	Résultat clé	Enseignement principal
MLP	$\approx 98,5\%$ accuracy (22 classes)	L'initialisation des poids peut faire ou défaire l'apprentissage (cas Xavier vs Constante)
CNN	$\approx 98,1\%$ accuracy (MNIST)	La convolution surpasse le MLP en exploitant la structure spatiale de l'image
LSTM/Seq2Seq	$MSE \approx 9 \times 10^{-4}$ (val.)	Le choix de cellule récurrente optimale dépend de la longueur de séquence ; le Beam Search perd son intérêt en régression continue

6.2 Limites et pistes d'amélioration

Plusieurs axes permettraient d'approfondir ce travail :

- **Partie I** : tester des architectures à plus forte capacité ou des méthodes d'ensemble (*Random*

Forest, Gradient Boosting) en comparaison directe avec le MLP sur ce jeu de données de taille modeste.

- **Partie II** : évaluer la robustesse du CNN face à des transformations (rotation, bruit, translation) et explorer l'augmentation de données pour améliorer encore la généralisation.
- **Partie III** : étendre l'horizon de prévision au-delà de 24h pour mieux révéler les avantages structurels du LSTM et du GRU sur les longues séquences, et explorer des mécanismes d'attention pour améliorer le Seq2Seq.

6.3 Mot de conclusion

Au-delà des résultats quantitatifs, ce projet illustre une démarche essentielle en science des données : le choix d'une architecture ne doit jamais être arbitraire, mais résulter d'une analyse fine de la structure des données et de la tâche à résoudre. Cette discipline méthodologique, appliquée ici à des problématiques agricoles concrètes, constitue un socle transférable à de nombreux autres domaines d'application du deep learning.

Références

- [1] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*. *Nature*, 323(6088), 533–536.
- [2] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-based learning applied to document recognition*. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [3] Glorot, X., & Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. *Proceedings of AISTATS*.
- [4] Ioffe, S., & Szegedy, C. (2015). *Batch normalization: Accelerating deep network training by reducing internal covariate shift*. *Proceedings of ICML*.
- [5] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A simple way to prevent neural networks from overfitting*. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- [6] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. *Neural Computation*, 9(8), 1735–1780.
- [7] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation*. *Proceedings of EMNLP*.
- [8] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to sequence learning with neural networks*. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization*. *Proceedings of ICLR*.
- [10] LeCun, Y., Cortes, C., & Burges, C. J. (1998). *The MNIST database of handwritten digits*. <http://yann.lecun.com/exdb/mnist/>
- [11] Ingle, A. (2020). *Crop Recommendation Dataset*. Kaggle. <https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>
- [12] Max Planck Institute for Biogeochemistry. *Jena Climate Dataset (2009–2016)*. Weather

Station Jena, Germany.

Annexes

A. Liste des fichiers produits

Fichiers générés par le projet

Fichier	Description
paxtze1_mtp. py	Script complet — Partie I (MLP)
best_mtp. ptfi	Poids du meilleur modèle MLP entraîné
teaxnzng_cuxsJes. png	Courbes d'apprentissage du MLP
con/uszon_matxzz. png	Matrice de confusion — 22 classes
znzt_compaxzson. png	Comparaison des initialisations Xavier / Gaussienne / Constante
paxtze2_cnn. py	Script complet — Partie II (CNN)
best_cnn. ptfi	Poids du meilleur modèle CNN entraîné
mnzt_samples. png	Échantillons du jeu de données MNIST
cnn_teaxnzng_cuxsJes. png	Courbes d'apprentissage du CNN LeNet
/eatuxe_maps_consJ1. png	Cartes de caractéristiques — Conv1
/eatuxe_maps_consJ2. png	Cartes de caractéristiques — Conv2
con/uszon_matxzz_cnn. png	Matrice de confusion — MNIST
mtp_sJs_cnn. png	Comparaison MLP vs CNN
paxtze3_tstm. py	Script complet — Partie III (LSTM/Seq2Seq)
best_seq2seq. ptfi	Poids du meilleur modèle Seq2Seq entraîné
xnn_compaxzson. png	Comparaison RNN / LSTM / GRU
decodzng_compaxzson. png	Comparaison décodage Glouton vs Beam Search
Cxop_xecommendatzon. cssJ	Jeu de données — Partie I
3ena_ctzmate_2oog_2o16. cssJ	Jeu de données — Partie III

B. Environnement technique

Bibliothèques principales utilisées

Bibliothèque	Usage
PyTorch	Construction et entraînement des réseaux
NumPy / Pandas	Manipulation des données
Scikit-learn	Prétraitement, métriques, split train/test
Matplotlib	Génération des figures et courbes

C. Reproductibilité

Tous les scripts fixent une graine aléatoire (`SEED = 42`) appliquée à PyTorch, NumPy et au module `random` de Python, garantissant la reproductibilité des résultats présentés dans ce rapport d'une exécution à l'autre sur un même environnement matériel.